Subscription Management System: Product Requirements Document

Product: Brightloop Subscription Platform | Stack: React, Python Django, PostgreSQL | Version 1.0 | 31 October 2026

1. Overview

1.1 Problem

Brightloop Software sells everything on subscription: software billed monthly or yearly, add-ons that customers switch on and off, and products with variants. Today subscriptions live in a spreadsheet, renewals are remembered by whoever notices first, invoices are typed by hand, payments are matched by scrolling bank statements, and revenue reports are built manually each quarter. Renewals are missed, invoices go out wrong, and nobody can say how much recurring revenue the company has.

1.2 Vision

One unified platform that runs the full life of a subscription business: set up products, plans, taxes and discounts; turn quotes into subscriptions; generate invoices automatically; record payments; and report revenue in one place. Customers manage their own subscriptions through a self-service portal.

1.3 Goals and success metrics

Goal	Metric / target
Eliminate manual invoicing	100% of recurring invoices generated by the system, zero typed by hand
No missed or duplicate billing	Exactly one invoice per subscription per billing period (enforced by a database constraint)
Accurate money handling	Tax, discount and totals calculated server-side using Decimal arithmetic, reproducible to the paisa
Visibility	Finance lead sees active subscriptions, revenue, outstanding and overdue in one dashboard

Goal	Metric / target
Performance	Every page and API call responds in under 2 seconds with thousands of subscriptions
Self-service	Customers view subscriptions and invoices without emailing anyone

1.4 Non-goals (version 1)

- Live payment gateway integration (card capture, UPI collection). Payments are recorded manually against invoices; the design leaves a clean hook for a gateway later.
- Multi-currency, multi-company and multi-language support.
- Dunning automation beyond overdue flags and reminder emails.
- Mobile native apps (the web app must be responsive).

2. Users and roles

Role	Who	Access
Admin	Owner of the system	Full access. Only role that can create staff, taxes, discount rules, products, plans and variants
Internal staff	Small day-to-day team	Create and manage customers, quotations, subscriptions, invoices and payments. Cannot manage users, taxes, discount rules or plan configuration
Customer	External buyer	Portal only: sees only their own subscriptions, invoices and payments

2.1 Permission matrix

Capability	Admin	Staff	Customer
Manage staff accounts	Yes	No	No
Manage products, variants, plans, taxes	Yes	Read only	No

Capability	Admin	Staff	Customer
Create discount rules	Yes	Read only	No
Manage quotation templates	Yes	Use only	No
Create and edit subscriptions	Yes	Yes	No
Generate, confirm, cancel, send, print invoices	Yes	Yes	Own invoices: view, download
Record payments	Yes	Yes	No
View dashboard and reports	Yes	Yes	No
Close, pause or renew a subscription	Yes	Yes	Own, only where the plan allows

Permissions are enforced on the backend (Django REST Framework permission classes and queryset scoping). Hiding a button in React is never the only protection.

3. Technology stack and architecture

Layer	Choice
Frontend	React 18 with Vite, React Router, TanStack Query for data fetching and caching, React Hook Form with Zod validation, Axios
Backend	Python 3.12, Django 5, Django REST Framework, SimpleJWT, django-filter, drf-spectacular (OpenAPI docs)
Database	PostgreSQL 16
Background jobs	Celery with Redis as broker, Celery Beat for scheduled billing and expiry jobs
PDF and email	WeasyPrint for invoice PDFs; Django email backend over SMTP
Deployment	Docker Compose (web, worker, beat, postgres, redis, nginx)

Layer	Choice
Testing	pytest and pytest-django, Vitest and React Testing Library

3.1 Architecture

A single-page React application talks to a Django REST API over JSON. Django apps are split by domain: `accounts`, `catalog` (products, variants), `plans`, `subscriptions`, `billing` (invoices, taxes, discounts), `payments`, `reports`. Business rules live in service modules, not in views or serializers, so the same logic is used by the API and the background jobs. Celery runs the daily billing job and the expiry job.

4. Functional requirements

4.1 Accounts, authentication and roles

- **AUTH-1 Sign-up.** A new customer creates an account with email and password. Email is unique across all accounts, compared case-insensitively. Duplicate sign-up returns a clear validation error.
- **AUTH-2 Password rule.** The password must be longer than eight characters (nine or more) and contain at least one uppercase letter, one lowercase letter and one special character. Validated on both the frontend (live checklist) and the backend (Django password validator).
- **AUTH-3 Login.** Email and password return a short-lived JWT access token and a refresh token. Passwords are hashed with Argon2 or PBKDF2.
- **AUTH-4 Forgot password.** The user enters their registered email; a single-use reset link valid for 60 minutes is emailed. The response is identical whether or not the email exists, to prevent account discovery. The new password follows AUTH-2.
- **AUTH-5 Roles.** Roles are Admin, Staff and Customer. Public sign-up always creates a Customer. Only an Admin can create a Staff member (name, email, temporary password or invite link).
- **AUTH-6 Customer scoping.** Every customer-facing endpoint filters data by the logged-in customer; requesting another customer's record returns 404.
- **AUTH-7 Profile.** Users can update their name and contact details and change their password.

4.2 Products and variants

The admin adds a product with: name, type (for example Software, Add-on, Service), sales price, cost price and notes. Margin (sales price minus cost price) is shown to admin and staff.

- **PROD-1 Variants.** A product can have variants defined by an attribute and value, each with an extra price. Example: attribute Brand, value Odoo, extra price 560. Effective unit price = sales price + variant extra price.
- **PROD-2 Default tax.** A product can carry a default tax, copied onto subscription lines.
- **PROD-3 Archive.** Products are archived, not deleted, once referenced by a subscription or invoice.
- **PROD-4 Validation.** Prices must be zero or positive. Cost price above sales price shows a warning, not an error.

4.3 Recurring plans

A plan defines what a subscription costs and how often it bills.

- **PLAN-1 Fields.** Name, price, billing period (daily, weekly, monthly, yearly), minimum quantity, available-from date and available-to date.
- **PLAN-2 Behaviour flags set by the admin per plan.** `auto_close` (the subscription closes by itself when it reaches its end), `closable` (can be closed early), `pausable` (can be paused), `renewable` (can be renewed).
- **PLAN-3 Availability.** A plan can only be selected on a new subscription between its available-from and available-to dates.
- **PLAN-4 Minimum quantity.** Subscription lines cannot go below the plan minimum quantity.
- **PLAN-5 Safe edits.** Changing a plan price affects only future subscriptions. Existing subscription lines keep their own unit price.

4.4 Quotation templates

- **TPL-1.** The admin creates a template with name, validity in days, a plan and a set of product lines (product, variant, quantity).
- **TPL-2.** A sales rep creates a quotation from a template. Plan and lines are copied in, and the quotation expiry is set to today plus the template validity days. Everything remains editable on the quotation.

- **TPL-3.** Expired quotations cannot be confirmed and are flagged automatically by the daily job.

4.5 Subscriptions

Each subscription has an auto-generated number (for example SUB-000123), a customer, a plan, a start date, an expiry date, payment terms and order lines. Each line holds product, variant, quantity, unit price, taxes and amount.

- **SUB-1 Lifecycle.** Draft, Quotation, Confirmed, Active, Closed, with allowed transitions in section 6. Anyone on the team can see the current stage as a visible status badge and a status timeline.
- **SUB-2 Payment terms.** Immediate, 15 days, 30 days, 45 days, or custom. Used to compute each invoice due date.
- **SUB-3 Next billing date.** The system keeps `next_billing_date` and advances it by the plan period each time an invoice is generated.
- **SUB-4 Pause, close, renew.** Allowed only when the plan permits it. Pausing stops invoice generation until resumed. Renewing extends the expiry date by one plan term. Closing early sets status Closed and records the reason and date.
- **SUB-5 Auto-close.** If the plan has `auto_close` and the expiry date passes, the daily job moves the subscription to Closed.
- **SUB-6 History.** Every status change and edit is recorded in an audit trail (who, what, when).

4.6 Invoicing

- **INV-1 Automatic generation.** A scheduled job runs daily. It finds Active, non-paused subscriptions whose `next_billing_date` is today or earlier and creates one invoice per due billing period, with customer details, lines, taxes, discounts and totals already filled in. No one types invoices by hand. Staff can also trigger generation for one subscription manually.
- **INV-2 Never twice, never skipped.** A unique database constraint on (subscription, period start, period end) makes duplicate invoices impossible. The job is idempotent, processes each subscription in its own transaction using `select_for_update(skip_locked=True)`, catches up on any periods missed during downtime, and logs failures per subscription so one error never blocks the rest.
- **INV-3 Snapshot.** Invoice lines copy the product name, price and tax rate at the time of billing, so later catalogue changes never alter past invoices.

- **INV-4 Status.** Draft, Confirmed, Paid, plus Cancelled as a terminal state. Overdue is not stored; it is derived (Confirmed, balance above zero, due date before today).
- **INV-5 Actions.** Confirm, cancel, send (emails the PDF to the customer) and print (downloads the PDF) from the invoice screen and from list bulk actions.
- **INV-6 Numbering.** Sequential, gap-free per year, for example INV-2026-000412, assigned on confirmation.
- **INV-7 Immutability.** Confirmed invoices cannot be edited. Corrections are made by cancelling and re-issuing.
- **INV-8 Billing-day scale.** Hundreds of due subscriptions must be processed in one run in a few minutes, using batched queries and bulk inserts of lines.

4.7 Taxes

- **TAX-1.** The admin defines tax rules: name, rate (percentage), active flag, and whether prices are tax-inclusive or exclusive.
- **TAX-2.** Taxes are attached to products and copied to lines; they are calculated automatically on every invoice with no manual entry.
- **TAX-3 Calculation order.** Line amount = quantity x unit price; apply discount; then tax on the discounted amount; invoice totals are the sum of line values. All arithmetic uses Decimal, rounded half-up to two places, stored as `numeric(12,2)`.
- **TAX-4.** Invoices show a tax breakdown per tax rule.

4.8 Discounts

Only the admin can create discount rules.

- **DISC-1 Fields.** Name, type (fixed amount or percentage), value, minimum purchase amount, minimum quantity, start date, end date, usage limit, times used, and scope.
- **DISC-2 Scope.** A discount applies to specific products, to specific subscriptions, or to all.
- **DISC-3 Auto-apply.** When a rule is eligible (dates valid, minimums met, usage remaining), it appears on the next invoice automatically as a discount line or line-level reduction, without anyone remembering it.
- **DISC-4 Stop conditions.** The discount stops after its end date or when the usage limit is reached. The usage counter is incremented in the same transaction that creates the invoice, so concurrent billing runs cannot exceed the limit.

- **DISC-5 Guardrails.** Percentage between 0 and 100; fixed discount never exceeds the line amount; one discount per line (the most valuable eligible rule wins); cancelling an invoice restores the usage count.
- **DISC-6 Visibility.** Applied discounts are shown on the invoice and on the subscription, with the rule name.

4.9 Payments

- **PAY-1 Recording.** Each payment records method (cash, bank transfer, card, UPI, cheque, other), amount, date and a note, linked to a specific invoice, plus who recorded it.
- **PAY-2 Partial payments.** Allowed. Outstanding balance = invoice total minus sum of payments.
- **PAY-3 Auto-paid.** When payments cover the total, the invoice moves from Confirmed to Paid automatically.
- **PAY-4 Validation.** Payment amount must be above zero and may not exceed the outstanding balance. Payments can only be recorded on Confirmed invoices.
- **PAY-5 Visibility.** A list of paid and outstanding invoices with filters (customer, status, date, overdue) so overdue invoices can be chased. Overdue rows are highlighted and show days overdue.
- **PAY-6 Reversal.** Only the admin can void a payment, which reopens the invoice balance and is audit-logged.
- **PAY-7 Reminders.** A daily job emails customers a reminder for invoices that are overdue (configurable cadence).

4.10 Customer portal

- **PORT-1.** Customers see only their own subscriptions (with status, plan, next billing date) and invoices (with status and balance).
- **PORT-2.** They can download invoice PDFs and see their payment history.
- **PORT-3.** They can close, pause or renew a subscription where the plan allows, with a confirmation step.
- **PORT-4.** They can update their profile and change their password.

4.11 Dashboard and reports

One place for the finance lead.

- **REP-1 KPI cards.** Active subscriptions, monthly recurring revenue (MRR), revenue collected in the period, total outstanding, overdue invoice count and amount.
- **REP-2 Views.** Revenue by month, subscriptions by status, revenue by plan, payments by method, and an overdue invoice list.
- **REP-3 Filters.** Date range, plan, customer and status, applied across all widgets.
- **REP-4 Summary view.** A one-page month-end summary that can be exported to CSV and printed.
- **REP-5 Performance.** Aggregates are computed in SQL with proper indexes and cached for 60 seconds in Redis; heavy lists are paginated server-side.

5. Data model

All tables use UUID or bigint primary keys, `created_at` and `updated_at`. Money columns are `numeric(12,2)`.

Entity	Key fields
User	email (unique, case-insensitive), password hash, name, role (admin, staff, customer), is_active
CustomerProfile	user, company name, phone, billing address, tax id
Product	name, type, sales_price, cost_price, notes, default_tax, is_active
ProductVariant	product, attribute, value, extra_price
Plan	name, price, billing_period, min_quantity, available_from, available_to, auto_close, closable, pausable, renewable, is_active
QuotationTemplate	name, validity_days, plan
QuotationTemplateLine	template, product, variant, quantity
Subscription	number (unique), customer, plan, status, start_date, expiry_date, next_billing_date, payment_terms, is_paused, closed_reason
SubscriptionLine	subscription, product, variant, quantity, unit_price, tax, amount

Entity	Key fields
Tax	name, rate, is_inclusive, is_active
Discount	name, type, value, min_purchase, min_quantity, start_date, end_date, usage_limit, times_used, scope
DiscountProduct, DiscountSubscription	link tables for scoped discounts
Invoice	number, subscription, customer, status, period_start, period_end, issue_date, due_date, subtotal, discount_total, tax_total, total, amount_paid
InvoiceLine	invoice, product name, quantity, unit_price, discount, tax_rate, tax_amount, amount
Payment	invoice, method, amount, payment_date, note, recorded_by, is_void
AuditLog	actor, entity type, entity id, action, before/after JSON, timestamp

5.1 Constraints and indexes

- Unique: User email (lower-cased), Subscription number, Invoice number, Invoice (subscription, period_start, period_end).
- Check constraints: prices at or above zero, payment amount above zero, discount percentage 0 to 100, expiry date after start date.
- Indexes: Subscription (status), (customer), (next_billing_date) partial where status is Active; Invoice (status, due_date), (customer), (subscription); Payment (invoice), (payment_date); AuditLog (entity type, entity id).

6. State machines

6.1 Subscription

From	To	Trigger
Draft	Quotation	Staff sends or saves as quotation
Quotation	Confirmed	Customer accepts; quotation not expired
Confirmed	Active	Start date reached or staff activates; first billing date set
Active	Closed	Closed early (if closable), or auto-closed at expiry (if auto_close)
Draft / Quotation	Closed	Cancelled or expired

Pause is a flag on an Active subscription (`is_paused`), not a separate status, so the five-stage lifecycle stays as specified. Renewal keeps the subscription Active and extends its expiry date.

6.2 Invoice

From	To	Trigger
Draft	Confirmed	Staff confirms; number assigned
Draft / Confirmed	Cancelled	Staff cancels (Confirmed with no payments only)
Confirmed	Paid	Payments cover the total

7. API outline

REST over JSON, versioned under `/api/v1/`, JWT bearer auth, OpenAPI docs generated automatically. List endpoints support pagination, search, ordering and filters.

Area	Endpoints
Auth	POST /auth/register, /auth/login, /auth/refresh, /auth/forgot-password, /auth/reset-password; GET and PATCH /auth/me
Staff (admin)	GET, POST /staff; PATCH /staff/{id}
Catalog	CRUD /products, /products/{id}/variants
Plans	CRUD /plans
Templates	CRUD /quotation-templates; POST /quotation-templates/{id}/create-quotation
Subscriptions	CRUD /subscriptions; POST /subscriptions/{id}/confirm, /activate, /close, /pause, /resume, /renew
Invoices	GET /invoices; POST /subscriptions/{id}/generate-invoice; POST /invoices/{id}/confirm, /cancel, /send; GET /invoices/{id}/pdf
Taxes and discounts	CRUD /taxes, /discounts (write: admin only)
Payments	GET, POST /invoices/{id}/payments; POST /payments/{id}/void
Portal	GET /portal/subscriptions, /portal/invoices, /portal/payments; POST /portal/subscriptions/{id}/close, /pause, /renew
Reports	GET /reports/summary, /reports/revenue, /reports/overdue, /reports/subscriptions

Errors use a consistent body: `{ code, message, fields }` with correct HTTP status codes (400, 401, 403, 404, 409).

8. Non-functional requirements

- **Performance.** Every page and API response under 2 seconds at p95 with at least 10,000 subscriptions and 100,000 invoices (verified with a seeded load-test dataset). Server-side pagination (default 25), `select_related` and `prefetch_related` everywhere, no

N+1 queries (checked with query-count tests), indexes from section 5.1, and caching for dashboard aggregates.

- **Security.** Argon2 hashing, HTTPS only, CORS limited to the frontend origin, rate limiting on login and reset, short-lived access tokens, CSRF-safe token handling, input validation on all endpoints, role checks on every view.
- **Reliability.** Billing job is idempotent and retry-safe; failed subscriptions are logged and retried the next run; database transactions around invoice, discount usage and next billing date.
- **Data integrity.** Decimal-only money, database constraints, soft archive instead of delete, audit trail on all financial changes.
- **Usability.** Responsive from 360 px wide; keyboard navigable; WCAG AA colour contrast; clear empty, loading and error states.
- **Observability.** Structured logs, job run history screen, error tracking hook.
- **Maintainability.** Linting (ruff, ESLint), type hints, at least 80% test coverage on billing, tax, discount and payment logic.

9. UI and design system

The supplied design system defines the visual language. It is a calm, plum-toned, card-based interface set in Inter.

9.1 Colour

Token	Hex	Use
Primary	#714B67	Primary buttons, active navigation, key actions, headers of key cards
Secondary	#A64D87	Accents, secondary progress, highlights, charts
Tertiary	#25232A	Inverted buttons, dark text, dark progress bars
Neutral	#F7F6F8	Page and card surfaces, input backgrounds

Each colour has a 12-step tint scale from near-black to white; use these steps for hover, disabled, borders and chart series. Destructive actions use a red (as on the delete icon button). Suggested status badge mapping: Draft grey, Quotation or Confirmed secondary tint, Active or Paid green, Closed or Cancelled dark neutral, Overdue red.

9.2 Typography

Inter for all three roles: Headline (large, regular weight, used for page titles and KPI numbers), Body (medium, for paragraph and table text) and Label (small, for field labels, badges and captions).

9.3 Components

- **Buttons.** Primary (filled #714B67, white text), Secondary (light neutral fill), Inverted (filled #25232A), Outlined (neutral border, transparent fill). Consistent rounded corners and label sizing.
- **Search field.** Neutral fill with a leading magnifier icon and the placeholder Search; present on every list screen.
- **Progress bars.** Thin rounded bars in primary, secondary and tertiary, used for usage limits (discounts), billing period progress and plan usage.
- **Navigation pill.** Rounded pill with icon buttons (home, search, profile); the active item is a filled primary circle. Used as the compact and mobile navigation.
- **Icon buttons.** Square grey edit (pencil) button, a primary labelled edit button, and round action buttons: magic wand (generate or automate, primary), shapes (variants or plans, secondary), tag (discounts, tertiary) and trash (delete, red).
- **Cards.** Light rounded cards on a neutral page; each module screen is built from cards for summary, filters and tables.

Implement the tokens as CSS variables (for example `--color-primary`) so the theme lives in one place.

9.4 Screens

Public: sign-up, login, forgot password, reset password. Admin and staff: dashboard, customers, products and variants, plans, quotation templates, subscriptions (list, detail with status timeline), invoices (list, detail, PDF preview), payments, taxes, discounts, staff management, reports. Customer portal: my subscriptions, my invoices, payment history, profile.

10. Delivery plan

Phase	Scope	Outcome
1. Foundation	Project setup, Docker, auth, roles, staff creation, design tokens, base layout	Users can register, log in, reset password; role-based shell
2. Catalogue	Products, variants, plans, taxes, quotation templates	Admin can launch a product and plan
3. Subscriptions	Quotation from template, subscription lifecycle, lines, audit trail	Staff can take a quote to Active
4. Billing	Invoice generation job, tax calculation, discounts, invoice actions and PDF	Billing day runs without manual work
5. Payments and portal	Payment recording, overdue tracking, reminders, customer portal	Money in is matched to invoices; customers self-serve
6. Reporting and hardening	Dashboard, filters, CSV export, performance tuning, load test, security pass	Finance lead gets one view; under 2 second target proven

11. Acceptance criteria (key flows)

- **Sign-up.** Given an existing email, when someone registers with it, then registration fails with a duplicate-email error. Given the password Abcdefg1, then it is rejected for missing a special character; given Abcdefg1!, it is accepted.
- **Staff creation.** Given a logged-in staff user, when they call the create-staff endpoint, then they get 403; an admin succeeds.
- **Variant pricing.** Given a product priced 1000 with variant Brand = Odoo adding 560, when added to a line, then the unit price is 1560.
- **Template.** Given a template valid for 30 days, when a quotation is created today, then its expiry is 30 days from today and lines match the template.
- **No double billing.** Given an Active subscription due today, when the billing job runs twice, then exactly one invoice exists for that period.

- **Tax.** Given a 1560 line with an 18 percent tax rule and no discount, then tax is 280.80 and the line total is 1840.80.
- **Discount.** Given a 10 percent discount with a limit of 5 uses, when the sixth eligible invoice is generated, then no discount is applied.
- **Payment.** Given a Confirmed invoice of 1000, when 400 and then 600 are recorded, then the invoice becomes Paid; a further payment is rejected.
- **Overdue.** Given a Confirmed invoice past its due date with a balance, then it appears in the overdue list and the dashboard overdue count.
- **Portal scoping.** Given customer A, when requesting customer B's invoice, then the response is 404.
- **Performance.** Given 10,000 subscriptions and 100,000 invoices, then list pages and dashboard load in under 2 seconds.

12. Assumptions, risks and open questions

12.1 Assumptions

- Single currency (INR) and a single company in version 1.
- Pause is modelled as a flag on Active subscriptions rather than a sixth status.
- Generated invoices start as Draft and staff confirm them; an optional setting can auto-confirm and auto-send.
- Payments are recorded manually; no gateway in version 1.

12.2 Risks and mitigations

Risk	Mitigation
Duplicate or skipped invoices on billing day	Unique constraint, idempotent job, per-subscription transactions, catch-up logic, tests with repeated runs
Rounding differences in totals	Decimal arithmetic, one documented rounding rule, unit tests on tax and discount math
Slow pages at scale	Indexes, pagination, query-count tests, cached aggregates, seeded load test

Risk	Mitigation
Permission leaks between customers	Queryset scoping in one base class, automated tests for every portal endpoint

12.3 Open questions

- Should discounts stack, or is one discount per line correct for Brightloop?
- Are taxes inclusive or exclusive of price by default?
- Should customers be able to start a new subscription from the portal, or only manage existing ones?
- Which reminder cadence do finance and support prefer for overdue invoices?